PROJECT HANDBOOK · OCTOBER 2026

Blackbox
The complete guide

Everything about how Blackbox was built — the tech stack and why each piece was chosen, the database, the AI interviewer, the group-discussion floor control, speech, eye-contact tracking and security — explained twice: once simply, once in proper technical terms. Read it end to end and you can answer any question about this project.

500 practice questions, 50 per domain	**5** target tracks that reshape the app	**4 + 1** AI panelists plus a moderator in GD	**0** video frames ever leave the browser

How to read this guide. Every topic has a yellow "**Explain like I'm 5**" box (the idea in plain words) and a blue "**In tech terms**" box (what you would say to an engineer). Green boxes are interview tips. Chapter 14 is a bank of likely questions with model answers.

Contents

1. What Blackbox is (the 60-second pitch)
2. The big picture: architecture
3. The tech stack, piece by piece, and why
4. How one request travels through the system
5. The database: PostgreSQL + Prisma
6. Sign-in and security
7. The AI layer: Gemini, fallbacks and prompts
8. The mock interview and the adaptive interviewer
9. Speech: listening and talking
10. Eye contact and posture: real measurements
11. The group discussion room: floor control, the queue and AI speakers
12. Practice Hub, dashboard, project review and the coach
13. Testing, quality and running the project
14. Interview question bank about this project
15. Glossary

1 What Blackbox is

EXPLAIN LIKE I'M 5

Blackbox is a practice gym for job interviews. You can talk to a pretend interviewer, join a pretend group discussion with four computer students, and study 500 flashcard-style questions. After every practice, it tells you what you did well, what to study, and how you looked on camera — like a coach who never gets tired.

IN TECH TERMS

Blackbox is a full-stack Next.js 16 web application for interview preparation, targeted at five tracks (AI Engineer, ML Engineer, Backend Architect, Full Stack Developer, Data Scientist). It provides LLM-driven mock interviews with adaptive follow-ups and per-answer scoring, a real-time group-discussion simulator with exclusive floor control, a curated 500-question practice bank stored in PostgreSQL, on-device computer vision for delivery metrics (MediaPipe), and browser/AI speech for voice interaction. Every feature degrades gracefully to deterministic offline logic when the AI provider is unavailable.

The main features

Feature	What it does
Mock interview	A 5-question interview with a company-styled interviewer. It reacts to your answers, coaches you when you say "I don't know", asks follow-ups about your projects, and scores every answer with key points to study.
Group discussion (GD)	A moderator plus four AI candidates with different personalities. Strict turn-taking: whoever speaks first holds the floor; everyone else waits in a queue.
Practice Hub	500 questions across 10 domains (50 each) with model answers, weak answers, key points and an AI coach.
Project review	Upload a project write-up and get interview questions about your own project, then get your answers reviewed.

Dashboard & performance	A readiness index, score trends, skills profile and recommendations, all reorganised around your chosen track.
Delivery analysis	Measures eye contact and posture from your webcam — on your device — and reports real percentages.

INTERVIEW TIP

Open with the problem, not the tech: “Students prepare for interviews alone, with no feedback. Blackbox simulates the real thing — including group discussions, which nobody can practise alone — and gives measured feedback.” Then mention two technical highlights: the floor-control state machine and the on-device eye-contact measurement.

2 The big picture: architecture

EXPLAIN LIKE I'M 5

Think of a restaurant. Your browser is the dining table where you sit. The Next.js server is the waiter who takes your order. PostgreSQL is the big notebook in the kitchen where everything is written down so nothing is forgotten. Gemini is a very smart chef in another building that the waiter phones for special dishes. Firebase is the security guard at the door who checks who you are. And the camera magic (MediaPipe) happens right at your table — your picture never goes to the kitchen.

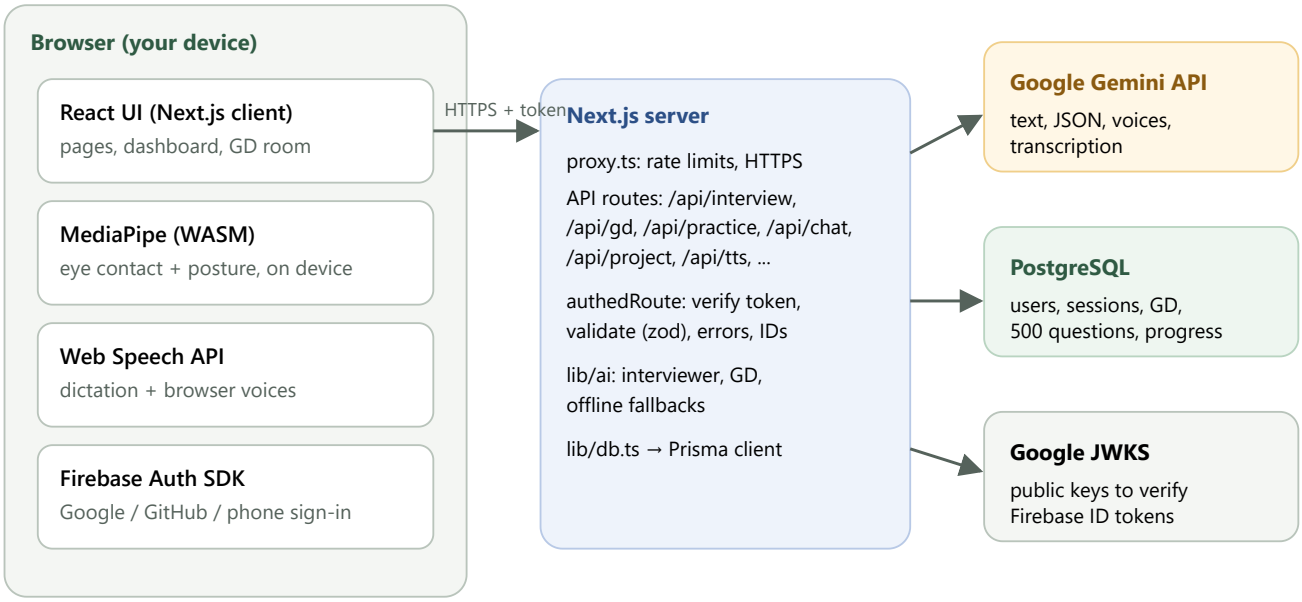


Figure 1 — Components and where they run. Video analysis and dictation run in the browser; data and AI calls go through the server.

IN TECH TERMS

- **Monolithic Next.js app** serving both the React UI (App Router, Server and Client Components) and a JSON API (Route Handlers under `app/api/*`). One deployable unit keeps development simple; boundaries inside the code (`lib/db.ts`, `lib/ai/*`, `lib/gd/*`) keep it modular.
- **Stateless API**: every request carries a Firebase ID token; the server verifies it and loads the user from PostgreSQL. No server sessions, so it scales horizontally.
- **Edge middleware** (`proxy.ts`): HTTPS redirect, per-IP rate limiting with separate budgets for AI routes and speech synthesis, and blocking of common probe paths.
- **AI is optional**: every AI call has a timeout and a deterministic fallback, so the product works with no API key.
- **Privacy by design**: computer vision runs in WebAssembly on the device; only summary percentages are stored.

Folder map

```
blackbox-ai/
├─ frontend/                ← the whole app (Next.js)
│  ├─ app/                  ← pages and API routes (App Router)
│  │  ├─ login/             ← landing + sign-in
│  │  ├─ (app)/             ← signed-in pages with the sidebar shell
│  │  ├─ (focus)/           ← full-screen rooms: interview session, GD room
│  │  ├─ (legal)/           ← terms, privacy
│  │  └─ api/               ← interview, gd, practice, chat, project, me, tts, transcribe,
health
│  ├─ components/           ← UI building blocks, motion, track switcher
│  ├─ hooks/useInterviewAnalyzer.ts ← eye contact + posture (MediaPipe)
│  ├─ lib/
│  │  ├─ ai/                ← gemini.ts, interview.ts, answer-signals.ts, gd.ts
│  │  ├─ gd/                ← floor-control.ts, conductor.ts, agents.ts, topic-packs.ts
│  │  ├─ server/            ← auth.ts (token checks), api.ts (route wrapper)
│  │  ├─ db.ts, prisma.ts   ← all database access
│  │  ├─ speech.ts          ← dictation, voices, recording
│  │  └─ validation.ts      ← zod schemas shared by client and server
│  ├─ prisma/               ← schema.prisma, migrations, seed + 500 questions
│  └─ scripts/dev-local.mjs ← one-command local database + server
├─ docs/                    ← this guide
└─ legacy/                  ← old code kept for reference
```

3 The tech stack, piece by piece, and why

For each piece: what it is in simple words, what it does in Blackbox, and why it was picked over alternatives. Interviewers love the “why”.

Technology	Simple meaning	Job in Blackbox	Why this choice
Next.js 16 (App Router, Turbopack)	A framework that builds websites with React and also runs a server.	All pages, routing, server rendering and the API routes.	One codebase for UI and API; server rendering for a fast landing page; file-based routing; route groups for different layouts. Alternatives (separate React + Express) mean two deployments and duplicated types.
React 19	A library for building screens out of reusable pieces.	Every screen and component.	Industry standard, huge ecosystem, Server Components reduce JavaScript sent to the browser.
TypeScript	JavaScript with labels on data, so mistakes are caught early.	All code, plus types generated from the database schema.	Catches bugs before running; safe refactors; the database, API and UI share types.
Tailwind CSS v4 + shadcn/Radix	Styling with small utility classes; ready-made accessible building blocks.	The design system: colour tokens, dark/light themes, dialogs, menus.	Fast, consistent styling; Radix gives keyboard and screen-reader support for menus and dialogs for free.
PostgreSQL	A very reliable database that stores data in tables.	Users, interview sessions, GD results, 500 questions, progress, coach chats.	Relational data with relationships and transactions; arrays and JSON columns; GIN indexes; free and runs everywhere. MongoDB was not needed because the data is relational.
Prisma 7	A translator between TypeScript code and the database.	Schema, migrations, type-safe queries, seeding.	Type-safe queries generated from one schema file; migrations are versioned SQL files; Prisma 7’s driver adapter (`@prisma/adapter-pg`) uses the standard `pg` driver.
Firebase Authentication	A ready-made, secure login service from Google.	Google, GitHub and phone sign-in.	Building password storage, OTP and OAuth safely is hard; Firebase is free at this scale. The server verifies tokens itself with `jose`, so it doesn’t need the Firebase Admin SDK.

Google Gemini API	A smart AI model we can ask questions over the internet.	Interview questions and reactions, grading, GD panelists, coach, project questions, natural voices, accurate transcription.	Strong quality with JSON mode, a generous free tier, plus text-to-speech and audio understanding in the same API. Called over plain REST with timeouts and model fallback.
Web Speech API	The browser's built-in ears and mouth.	Live dictation of your answers; reading questions aloud.	Free, no server cost, works instantly in Chrome and Edge. Upgraded by Gemini voices and transcription when a key is set.
MediaPipe Tasks Vision	Google's camera AI that finds your face and body in a video.	Eye contact, posture and smile measurement.	Runs in the browser via WebAssembly at ~7 frames/second analysed — fast, free and private (video never uploaded).
zod	A gatekeeper that checks data has the right shape.	Validates every API request and every AI JSON reply.	One schema gives both runtime checks and TypeScript types; rejects bad input with clear 422 errors.
jose	A small library to check digital signatures on tokens.	Verifies Firebase ID tokens against Google's public keys.	Lightweight, standards-based (JWT/JWKS), works in any runtime.
Recharts	Ready-made charts for React.	Score-trend chart on the dashboard.	Declarative charts that follow the design tokens.
Vitest	A tool that runs automatic checks on code.	Tests for floor control, the discussion conductor and answer reading.	Fast, TypeScript-native, supports fake timers for timing-heavy logic.
embedded-postgres / Docker	A way to run a database on your laptop without installing it.	`npm run dev:local` starts a private PostgreSQL inside the project.	One command to run everything in VS Code; Docker Compose is provided as an alternative.

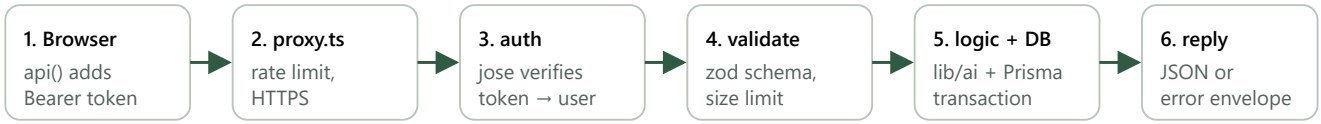
INTERVIEW TIP

If asked "why not microservices?": the team and traffic are small, so a modular monolith gives faster development and simpler operations; the code is already split into clear modules (`lib/ai` , `lib/gd` , `lib/db`) that could become services later if needed.

4 How one request travels through the system

EXPLAIN LIKE I'M 5

When you press "Submit answer", your browser writes a letter with your answer and your ID badge. A doorman checks you aren't sending letters too fast. A guard checks the badge is real. A clerk checks the letter is filled in properly. Then the interviewer reads it, writes it in the notebook, thinks of a reply, and sends a letter back.



Any failure stops the chain and returns a consistent error: { error: { code, message, requestId } } with the right HTTP status (401, 413, 422, 429, 500...).

Figure 2 — The request pipeline, implemented by `authedRoute()` in `lib/server/api.ts`.

IN TECH TERMS

1. **Client**: `lib/api-client.ts` attaches `Authorization: Bearer <Firebase ID token>`, sets a timeout with `AbortSignal.timeout`, and retries once with a refreshed token on `TOKEN_EXPIRED`.
2. **Edge**: `proxy.ts` applies a sliding-window rate limit per IP: 120 requests/min for general API traffic, 20/min for AI-generating POSTs, 60/min for speech synthesis.
3. **Authentication**: `requireUser()` verifies the JWT signature, issuer, audience and expiry against Google's JWKS using `jose`, then upserts the user row (first request provisions the account).
4. **Validation**: `parseBody(req, schema, maxBytes)` enforces a body size limit (413) and the zod schema (422 with field details).
5. **Business logic** in `lib/ai/*` and data access only through `lib/db.ts`.
6. **Errors**: known `ApiError`s map to status codes; anything unexpected becomes a generic 500 and is logged with a request ID that is also returned to the client.

5 The database: PostgreSQL + Prisma

EXPLAIN LIKE I'M 5

A database is a set of super-organised notebooks. One notebook lists every user. Another lists every interview, and each interview page says which user it belongs to. When you delete your account, every page with your name is torn out too, so nothing is left behind.

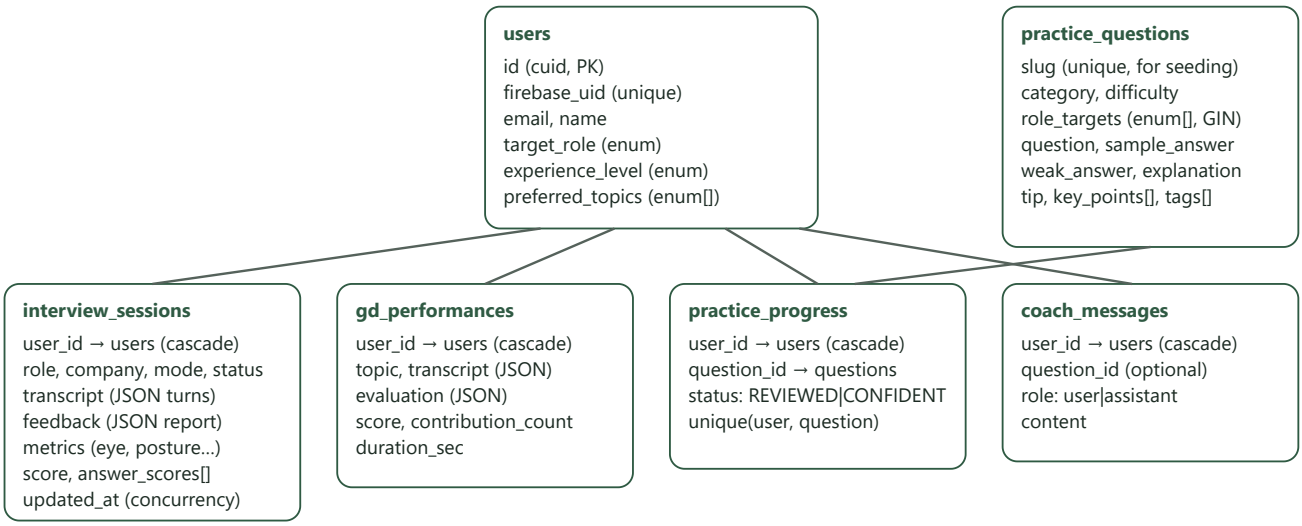


Figure 3 — Tables and relationships (simplified). Every per-user table deletes with its user (`onDelete: Cascade`).

IN TECH TERMS

- **Schema-first**: `prisma/schema.prisma` defines models and enums (`TargetRole` , `ExperienceLevel` , `PracticeCategory` , `Difficulty` , `InterviewMode` , `SessionStatus` , `PracticeStatus` , `ChatRole`). `@@map` gives snake_case table and column names.
- **Migrations**: versioned SQL in `prisma/migrations` (`init` , then `practice_key_points` which added a `TEXT[]` column). Applied with `prisma migrate deploy` .
- **Postgres features used**: enum types, array columns (`role_targets` , `tags` , `key_points`), a **GIN index** on `role_targets` so "questions for my track" (`has` / `isEmpty`) is fast, JSONB for transcripts and reports, a unique constraint on `(user_id, question_id)` for progress upserts.
- **Optimistic concurrency**: interview turns are appended with `updateMany where { id, updatedAt: expected }` . If two submits race, only one write matches the old timestamp; the other gets a 409 instead of corrupting the transcript.
- **Seeding**: `prisma/seed.ts` upserts the 500 questions by `slug` in batches of 25, so it is idempotent (safe to run on every deploy).
- **Connection**: one Prisma client singleton with the `PrismaPg` adapter (connection pooling via `pg`).

Why JSON columns for transcripts?

A transcript is always read and written as a whole and its shape evolves (turn kinds, key points, refined flags were added later). Storing it as JSONB avoids a join per turn and avoids migrations for every new field, while the fields we filter and aggregate on (score, status, role, dates) stay as proper typed columns.

6 Sign-in and security

EXPLAIN LIKE I'M 5

Google gives you a special sticker after you log in. Every time you ask Blackbox for something, you show the sticker. Blackbox checks the sticker with Google's official stamp pad to make sure it isn't fake or old. It also only lets you open your own notebook pages, never someone else's.

IN TECH TERMS

- **Firebase Auth** on the client (Google, GitHub OAuth; phone OTP with reCAPTCHA). The client obtains a short-lived **ID token** (a signed JWT, ~1 hour).
- **Server verification** with `jose.jwtVerify` against `securetoken@system.gserviceaccount.com` JWKS: checks RS256 signature, `iss`, `aud` (project ID) and expiry. Keys are cached.
- **Authorization**: every query is scoped by the user ID, e.g. `getOwnedInterview(userId, id)` — you can't read another user's session even with its ID (prevents IDOR).
- **Demo mode** for local development: a `dev:<id>` token accepted only when `DEV_AUTH_BYPASS=true` *and* not in production.
- **Hardening**: security headers (CSP, HSTS, X-Frame-Options, Referrer-Policy, Permissions-Policy, COOP/CORP), rate limits, body size limits, strict zod validation, generic 500 messages, no secrets in code (env vars), the Gemini key sent in a header (not the URL) so it never appears in logs.
- **Privacy**: account deletion (`DELETE /api/me`) removes the user and, via cascades, all their data. Video is processed on-device; recorded answer audio (only with Gemini on) is transcribed in memory and never stored.

7 The AI layer: Gemini, fallbacks and prompts

EXPLAIN LIKE I'M 5

Blackbox has a very clever helper (Gemini) it can phone. But phones sometimes don't connect. So Blackbox also keeps a big book of good questions and rules for scoring, and if the phone call fails, it uses the book. You still get a full interview — just a bit less magical.

IN TECH TERMS

- **REST client** (`lib/ai/gemini.ts`): `generateText` and `generateJson` call `generativelanguage.googleapis.com/v1beta/models/{model}:generateContent` with the key in the `x-goog-api-key` header.
- **Resilience**: hard timeout per call; one retry on 429/5xx; on 404 (retired model) it falls through a `GEMINI_MODELS` list and remembers the model that worked; "thinking" is kept low for latency.
- **Structured output**: JSON mode plus a **zod schema** validates every AI reply; malformed JSON is treated as "AI unavailable", so the fallback runs.
- **Graceful degradation**: every feature catches `AIUnavailableError` and uses deterministic logic: practice-bank questions, a heuristic rubric, topic packs for GD, generic project questions.
- **Extra capabilities**: `synthesizeSpeech()` (Gemini TTS models, prebuilt voices like Charon, Kore, Puck; PCM wrapped into WAV) and `transcribeAudio()` (16 kHz WAV sent inline for accurate transcripts).
- **Prompt design**: system prompt sets persona, company style, track focus, level and mode; rules demand plain spoken English (no markdown), one question per turn, and JSON with a fixed schema. Server-side guards still enforce rules the model might ignore (e.g. follow-up limits, on-topic check in GD).

INTERVIEW TIP

"What if the LLM is down or returns garbage?" — Every call has a timeout, the JSON is schema-validated, and each feature has an offline path that produces a complete, coherent session. The health endpoint reports `ai: "configured" | "fallback"` and the UI labels heuristic scores honestly.

8 The mock interview and the adaptive interviewer

EXPLAIN LIKE I'M 5

The interviewer doesn't just read a list. It listens. If you say "I don't know", it gives you a few hints and lets you try again, like a kind teacher. If you talk about something you built, it gets curious and asks about it. If your answer is short, it nudges you to say more. At the end, it tells you exactly which ideas to study for next time.

The flow of one interview

1. **Setup**: choose company style (Google, Amazon, Microsoft, Meta, AI lab, startup, consulting), mode (Coached, Standard, Bar raiser), experience level and voice.
2. **Preflight**: camera check, accent selection for dictation, AI voice availability.
3. **Opening**: the interviewer greets you and asks you to introduce yourself.
4. **Loop**: the interviewer speaks → you answer (voice or typing) → the server reads your answer and decides the next move → repeat. 5 main questions, plus up to 2 follow-ups and coaching retries.
5. **Finish**: the evaluation report with scores, strengths, improvements, a practice plan and an answer-by-answer review with *key points to study*.

How the interviewer decides what to say next

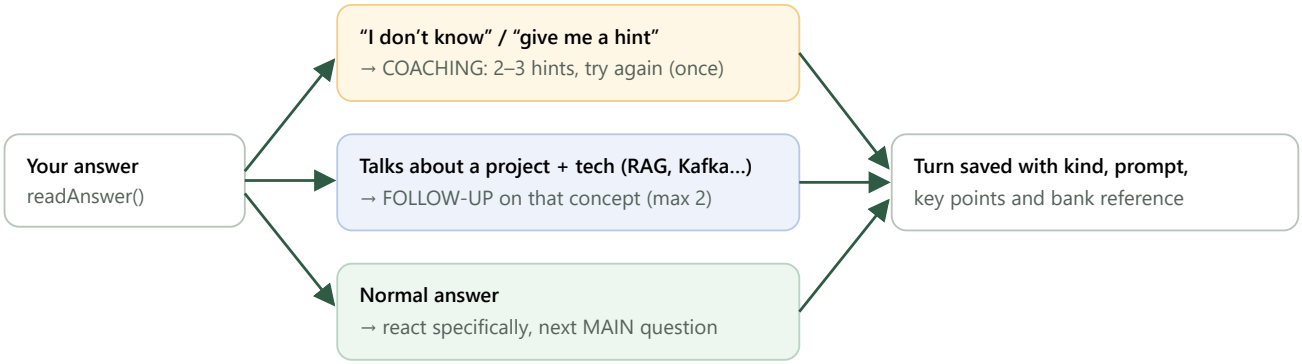


Figure 4 — The decision made after every answer (`lib/ai/interview.ts → nextTurn`).

IN TECH TERMS

- **Answer signals** (`lib/ai/answer-signals.ts`): regular expressions detect "don't know", help requests, project stories and nervousness; a dictionary of ~28 technologies (RAG, vector DBs, LoRA, Kafka, Redis, Kubernetes, React, SQL…) maps each to a realistic follow-up question and study points. Unit-tested.
- **Allowed moves** are computed on the server: coaching only once per question; follow-ups only after a project story, max 2, never twice in a row; main questions until 5. The model chooses within those moves (JSON: `kind, reaction, question, keyPoints`), and the server overrides anything not allowed.
- **Personalisation**: experience level maps to difficulty (Entry/Junior → Easy/Medium, Mid → Medium/Hard, Senior/Staff → Hard); the profile's preferred topics are asked first; tone follows the mode and the candidate's energy.
- **Turn model**: each interviewer turn stores `content` (what was said), `prompt` (the bare question), `kind` (question / followup / coaching), `keyPoints` and `ref` (practice-bank ID).
- **Idempotency**: the client sends the interviewer-turn number with every answer. A duplicate or late submit for an old turn gets 409 `NOT_YOUR_TURN` , and optimistic concurrency stops two answers being written at once.
- **Scoring without AI** (heuristic rubric): `30 + length×30 + structure×15 + impact×15 + ownership×10 − filler penalty` , clamped 15–92. Length is best at 90–260 words; structure counts signposts ("first", "because", "trade-off"); impact counts numbers and outcome verbs; ownership counts "I built / I decided". "Don't know" answers are capped at 12 and marked *Not attempted*.
- **Overall score** = technical depth 30% + problem solving 25% + communication 20% + structure 15% + composure 10%.

9 Speech: listening and talking

EXPLAIN LIKE I'M 5

Your browser has ears and a mouth. The ears write down what you say while you speak. We tell the ears your accent (like "English, India") so they understand you better. The mouth reads questions aloud; we pick the nicest-sounding voice and make it pause between sentences like a real person. If Blackbox has its Gemini key, an even better mouth and a second, more careful pair of ears are used.

Listening (speech-to-text)

IN TECH TERMS

- **Web Speech API** `SpeechRecognition` with `continuous = true` and `interimResults = true`: interim words stream into the answer box as you speak; final words replace them.
- **Accent**: `rec.lang` is the candidate's accent (en-IN, en-US, en-GB…), defaulting to the browser locale — a major accuracy improvement for Indian English.
- **Restarts**: browsers end a session after silence or ~60 s; the recognizer commits what it heard and restarts immediately in `onend`, so long answers aren't cut off.
- **Accurate transcript (with Gemini)**: while you answer, `AnswerRecorder` captures microphone audio with the Web Audio API, downsamples to 16 kHz mono WAV in the browser, and on submit sends it to `/api/transcribe`. Gemini transcribes it, given your accent and the question for spelling technical terms. The refined text replaces the live text and is flagged `refined`.
- **Browser note**: Chrome streams words live; Edge returns words only after pauses — the UI tells Edge users.

Talking (text-to-speech)

IN TECH TERMS

- **Voice ranking** `voiceQuality()`: neural voices (names containing "Natural/Online/Neural/Premium") score highest, then Google voices; legacy Windows SAPI voices (David, Zira) score lowest; the user's accent region gets a bonus.
- **Natural delivery**: text is split into sentences (very long ones at a comma), each spoken as its own utterance with a slightly varied rate (±3%) and a pause of ~260–500 ms between sentences (longer after questions). This removes the "machine reading a book" effect and avoids Chrome cutting off long utterances.
- **Distinct speakers**: in the GD room each panelist gets a different voice (picked without repeats) plus its own pitch/pace.
- **Gemini voices** (when configured): `/api/tts` returns natural speech from Gemini's TTS models with a delivery style, e.g. "warmly and conversationally". Falls back to the browser voice on any error.
- **Captions**: the sentence being spoken is shown on screen with a blur-in animation.

10 Eye contact and posture: real measurements

EXPLAIN LIKE I'M 5

A tiny camera brain on your computer finds 478 dots on your face and dots on your shoulders, about 7 times every second. It checks: is your head facing the screen, and are your eyes looking at it? Are your shoulders level and is your head not sinking down? It adds up the seconds you did it right. If you looked at the screen for 3 minutes out of 4, your eye contact is 75%. Nothing is guessed, and your video never leaves your computer.

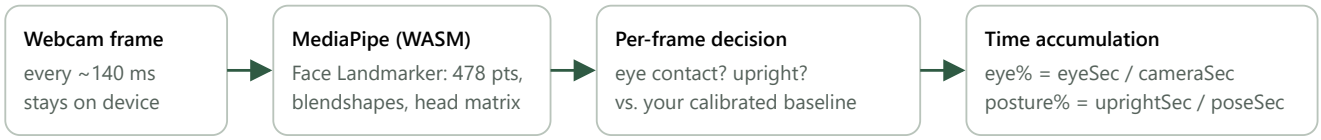


Figure 5 — The measurement pipeline in `hooks/useInterviewAnalyzer.ts`.

IN TECH TERMS

Eye contact

- **Head pose**: from MediaPipe's 4×4 facial transformation matrix (column-major), yaw = asin(−r_{20}) and pitch = atan2(r_{21}, r_{22}).
- **Gaze**: the blendshapes `eyeLookIn/Out/Up/Down` (0–1) tell whether the eyes are turned away; above 0.55 counts as looking away.
- **Calibration**: the first ~14 steady frames (~2 s) set the candidate's own baseline yaw/pitch, because webcams sit at different heights. Afterwards a frame counts as eye contact when |yaw − base| ≤ 16°, |pitch − base| ≤ 14° and gaze isn't away.
- **Blinks** (both `eyeBlink` > 0.6) keep the previous state, so blinking isn't penalised.
- **Score** = seconds of eye contact ÷ seconds of camera analysis. Leaving the frame counts against you; face presence is reported separately.

Posture

- Pose Landmarker gives nose (0) and shoulders (11, 12) with visibility scores; frames only count when both shoulders are visible (> 0.5).
- **Scale-free features**: shoulder width; neck ratio = (shoulder mid-height − nose height) ÷ shoulder width; shoulder tilt angle.
- After calibrating a baseline, a frame is *not upright* if tilt > 7° (uneven shoulders), neck ratio < 78% of baseline (slouching / head sinking) or shoulder width > 130% of baseline (hunched too close).
- **Score** = upright seconds ÷ seconds shoulders were visible; reported only with ≥ 10 s of visible shoulders, otherwise "Not measured".

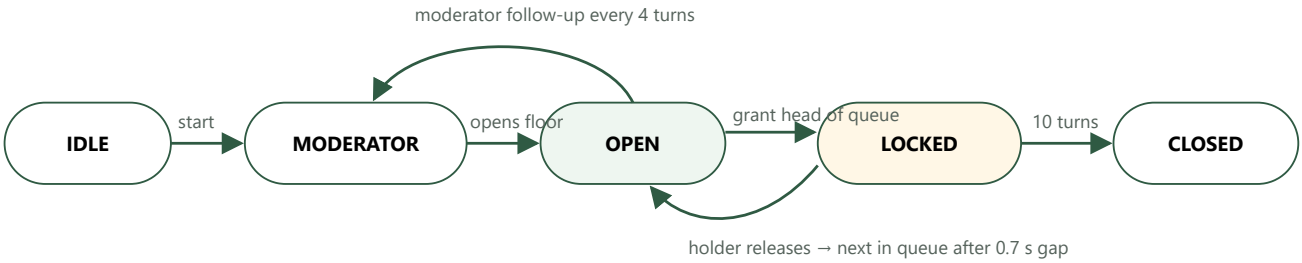
Honesty rules

- Time is accumulated per processed frame (capped at 0.5 s per frame so a frozen tab doesn't inflate numbers).
- Metrics are sent only if at least 15 s of camera time was analysed; the results page shows exactly how they were measured (e.g. "looking at the screen for 2m 31s of 3m 14s").
- MediaPipe's harmless "INFO: Created TensorFlow Lite XNNPACK delegate" console lines are filtered so they don't appear as errors.

11 The group discussion room: floor control, the queue and AI speakers

EXPLAIN LIKE I'M 5

Imagine a talking stick. Only the person holding the stick may talk. The teacher (moderator) holds it first and asks a question, then puts the stick on the table — now anyone may reach for it. Whoever reaches first gets it, and everybody else must wait in a line. Nobody can shout over the person holding the stick. When they finish, the stick goes to the next person in line. That is exactly how Blackbox's GD room works.



While MODERATOR or LOCKED: everyone else is WAITING and requests are rejected. While OPEN: requests join the queue in arrival order.

Figure 6 — The floor-control state machine (`lib/gd/floor-control.ts`).

IN TECH TERMS

FloorController — the lock

- A pure TypeScript **state machine** (no timers, no I/O): `IDLE → MODERATOR → OPEN ⇄ LOCKED → CLOSED`.
- One **exclusive lock**. In `OPEN`, `request(id)` appends to a FIFO **queue**; in `LOCKED` / `MODERATOR`, requests are rejected with a reason (`locked`, `not-open` …), so nobody can interrupt.
- **Lock tokens**: every grant increments a token. `release(token)` only works for the current holder's token — a slow AI reply arriving late can never release or speak under a lock it no longer owns (prevents race conditions).
- Exposes an immutable snapshot with `subscribe/getSnapshot`, used by React's `useSyncExternalStore` for tear-free rendering.

DiscussionConductor — the orchestration

- Runs the session on top of the lock: moderator intro → opens the floor → collects requests during a **1.4 s bidding window** → grants the first requester → hand-off to the next queued participant after a **0.7 s gap**.
- **AI bidding**: each agent has an *eagerness* (0.45–0.75); agents bid after a random 0.9–3.1 s delay, so the human has a fair chance to grab the floor first.
- **Moderator**: a follow-up question every 4 turns; nudges after 7 s of silence; closes after 10 turns.
- **Watchdogs**: a user turn is capped at 120 s and an AI turn at 45 s; every async step uses an `AbortController` so leaving the room cancels everything cleanly.
- All I/O (model calls, speech, timers, randomness) is **injected**, so the whole flow is unit-tested deterministically with Vitest fake timers.

The multi-AI speakers

Panelist	Personality	How they talk
Alex Chen	Analytical	Frames the decision, backs points with simple facts; agrees or disagrees with the last speaker by name.
Maya Patel	Pragmatic	"In practice…" — what really happens in teams and companies.
Ryan Blake	Contrarian	Politely takes the opposite side of whoever spoke last.
Sam Kim	Synthesizer	Names who said what, finds common ground, proposes a middle path.
Dr. Raj Mehta	Moderator	Introduces the topic, asks follow-ups, nudges and closes.

- **With Gemini**: the prompt demands everyday words, short sentences (mostly under 15 words), 2–3 sentences max, reacting to the previous speaker by name, and staying strictly on the moderator's current question. An **on-topic guard** rejects replies that share fewer than two content words with the topic and recent discussion, replacing them with a grounded line.
- **Without Gemini**: each of the 15 topics has a **topic pack** of plain-language pro, con and middle-ground points. Lines are composed from the persona, the stance of the previous speaker (detected by matching the pack or keywords), points not yet used, and natural openers ("Hmm, I'm not fully convinced…", "On that question…").

- **Evaluation**: clarity 30%, relevance 30%, collaboration 20%, initiative 20%. Offline, contributions are scored on length, structure words, topic relevance and building on others, and the score is capped unless you spoke at least three times — panels reward consistent presence.

12 Practice Hub, dashboard, project review and the coach

Practice Hub — 500 questions

EXPLAIN LIKE I'M 5

It's a library of 500 interview flashcards in 10 shelves: LLMs, ML, MLOps, System Design, DSA, Backend, Frontend, Analytics, Behavioural, Communication. Each card shows the key points, a great answer, a weak answer, why it matters and a tip. You mark cards as "reviewed" or "confident", and the app remembers.

IN TECH TERMS

- Questions live in PostgreSQL (seeded from TypeScript files under `prisma/seed-data`; a check script verifies 50 per category, no duplicate slugs or texts, 3–4 key points each).
- `GET /api/practice`: filtered by the user's track with `roleTargets has role OR isEmpty` (GIN index), by category, difficulty and search; **cursor pagination**; facet counts; per-user progress joined in.
- Progress is an upsert on the unique `(user, question)` pair; an AI grader reviews a typed attempt; the per-question **coach** chat (`/api/chat`) is persisted in `coach_messages`.

Dashboard and track

- **Readiness index** = weighted average of available signals: interview average (50%), practice coverage of your track's core areas (30%; "confident" counts fully, "reviewed" 60%), group-discussion average (20%). Missing signals are left out and the weights renormalised.
- **Track**: chosen at onboarding and switchable anytime from the dashboard header; it is saved to your profile and reshapes the question mix, interview focus, competencies, module order and recommendations.

Project review

Paste or upload a project write-up; the server asks Gemini for 3–8 tailored questions (or uses six proven generic ones offline), then reviews each of your answers. Text extraction from uploaded files happens in the browser.

13 Testing, quality and running the project

QUALITY GATES

- `npm run typecheck` — TypeScript strict mode
- `npm run lint` — ESLint incl. React hooks rules
- `npm test` — Vitest: floor control, conductor, answer signals
- `npm run build` — production build of every route
- `npx tsx scripts/check-question-bank.ts` — bank integrity

RUN IT LOCALLY

1. Open the folder in VS Code.
2. Terminal: `cd frontend`
3. `npm install` (first time)
4. `npm run dev:local`
5. Open `http://localhost:3000` → demo account.

Add `GEMINI_API_KEY` in `frontend/.env.local` for AI voices, accurate transcripts and model grading.

Deploying to production

1. Provision PostgreSQL (e.g. Neon, Supabase, RDS) and set `DATABASE_URL`.
2. Set Firebase variables, `GEMINI_API_KEY`, `NEXT_PUBLIC_SITE_URL`; add the domain to Firebase authorised domains.
3. Release step: `npm run db:deploy && npm run db:seed`; then `npm run build && npm start` (or Vercel).
4. Point uptime checks at `/api/health`. Use a shared store (e.g. Redis) for rate limits if running several instances.

14 Interview question bank about this project

Short, confident answers you can give when someone asks about Blackbox.

> **Q1. Give me a two-minute overview of your project.**
>
> Blackbox is an interview-preparation platform for engineers. It has an adaptive AI mock interviewer, a group-discussion simulator with four AI candidates and strict turn-taking, a 500-question practice bank, and on-device webcam analysis for eye contact and posture. It's a Next.js 16 app with a PostgreSQL database through Prisma, Firebase sign-in verified on the server, and Google Gemini for AI — with deterministic fallbacks so it works even without the AI. The parts I'm proudest of are the floor-control state machine for discussions and making the delivery metrics genuinely measured rather than estimated.

> **Q2. How does the group discussion stop people from interrupting each other?**
>
> A finite state machine with a single exclusive lock. When someone holds the floor the state is LOCKED and every other request is rejected; when the floor is OPEN, requests join a FIFO queue and are granted one at a time with a short hand-off gap. Each grant has an increasing token, and only the current token can release the lock, so a late AI response can't steal or release the floor.

> **Q3. How do you queue participants, and is it fair to the human?**
>
> Requests made during a short bidding window after the floor opens are queued in arrival order. AI agents bid after a random 0.9–3.1 second delay weighted by their eagerness, so a human who speaks up promptly usually gets the floor first, like in a real GD.

> **Q4. How do multiple AI speakers stay distinct and on topic?**
>
> Each agent has a persona (analytical, pragmatic, contrarian, synthesizer), its own voice, pitch and pace, and its own Gemini voice when available. The prompt forces plain, short, on-topic replies that react to the previous speaker; a server-side check rejects off-topic replies. Offline, lines are built from hand-written topic packs of pro, con and middle-ground points.

> **Q5. How do you calculate eye contact? Isn't it just a dummy number?**
>
> No, it's measured. MediaPipe's face landmarker runs in the browser about seven times a second. Each frame I compute head yaw and pitch from the face transformation matrix and check the eye-gaze blendshapes, compared to a baseline calibrated in the first two seconds. Eye contact is the seconds looking at the screen divided by the seconds analysed. Posture works the same way with shoulder tilt, neck height relative to shoulder width, and distance. Video never leaves the device.

> **Q6. What happens if Gemini is down or you have no API key?**
>
> Every AI call has a timeout and schema validation, and every feature catches the failure and uses deterministic logic: questions from the practice bank filtered by track and level, a transparent heuristic rubric, topic packs for discussions, and generic project questions. The UI labels heuristic scores honestly.

Q7. How does the interviewer adapt to the candidate?

After each answer the server reads signals — "don't know", help requests, project mentions with specific technologies, nervousness. Then it chooses from allowed moves: coach with hints and allow one retry, ask a follow-up about the project's technology, or acknowledge specifically and move on. Difficulty follows experience level and the profile's focus areas come first. The model chooses within those rules and the server enforces them.

Q8. Why PostgreSQL and not MongoDB?

The data is relational: users own sessions, progress links users and questions, and I need constraints, transactions and cascading deletes. Postgres also gives JSONB for transcripts, arrays and GIN indexes for track filtering — so I get document flexibility where I need it without giving up relational guarantees.

Q9. How do you prevent duplicate answer submissions?

Two layers. The client sends the interviewer-turn number it is answering; a mismatched number returns 409. And writes use optimistic concurrency: the update only applies if the row's updated-at timestamp is unchanged, so two concurrent submits can't both be written.

Q10. How is authentication secured?

Firebase handles sign-in; the client sends the ID token on every request; the server verifies the JWT's RS256 signature, issuer, audience and expiry with Google's public keys using jose. Every query is scoped to the user's ID, so even guessing another session's ID returns 404.

Q11. How did you make speech recognition more accurate?

Three things: setting the recognizer to the candidate's accent such as en-IN, restarting immediately when the browser ends a session so words aren't lost, and — when Gemini is configured — recording the answer as 16 kHz WAV in the browser and getting an accurate server transcription that also knows the question, so technical terms are spelled correctly.

Q12. Why does the AI voice sound more natural?

I rank voices by quality, preferring neural voices, and speak sentence by sentence with small varied pauses and slight rate variation instead of one flat utterance. With Gemini, I use its TTS voices with a speaking style prompt.

Q13. How is the readiness index calculated?

A weighted average of the signals that exist: interview average 50%, practice coverage of the track's core areas 30%, group-discussion average 20%; missing signals are excluded and weights renormalised, so new users aren't penalised for features they haven't tried.

Q14. How do you protect the API from abuse and cost spikes?

Per-IP rate limits in the edge proxy with a stricter budget for AI routes, body-size limits, strict validation, and timeouts on model calls. The Gemini key is only used server-side and sent in a header.

Q15. How did you test timing-heavy code like the discussion room?

The conductor receives all its dependencies — model calls, speech, clock, randomness — through injection, so tests drive it with Vitest fake timers and fake agents and assert exact sequences: who gets the floor, that interruptions are rejected, hand-offs, and the transcript at the end.

Q16. What would you improve next?

A shared rate-limit store for multi-instance deploys, real-time multiplayer GD with WebSockets so real candidates can join, server-side streaming of model replies to cut perceived latency, and analytics on which questions candidates struggle with to personalise practice.

Q17. How do you handle user privacy?

Video analysis runs on-device and only percentages are stored; recorded answer audio is transcribed in memory and not stored; users can delete their account and all data in one action thanks to cascading foreign keys; and the privacy page documents exactly which processors are used.

Q18. Why Next.js instead of a separate frontend and backend?

One codebase and one deployment, shared TypeScript types and zod schemas between client and server, server rendering for a fast landing page, and route handlers that are enough for this API. It’s a modular monolith that could be split later.

15 Glossary

Term	Like I'm 5	In tech terms
API	A menu of things one program can ask another to do.	A defined set of HTTP endpoints with request/response formats.
JWT / ID token	A signed sticker proving who you are.	A base64 JSON token signed with RS256; verified with the issuer's public keys (JWKS).
State machine	Rules saying which "mode" comes after which.	A finite set of states with explicit allowed transitions.
Mutex / lock	The talking stick.	Mutual exclusion: only one holder at a time.
Queue (FIFO)	A line — first come, first served.	First-in, first-out ordering of pending requests.
Race condition	Two people grabbing the same toy at once.	Outcome depends on timing of concurrent operations; prevented with tokens and atomic writes.
Optimistic concurrency	"Only save if nobody changed it since I looked."	Conditional update on a version/timestamp; conflicts return 409.
Migration	Instructions to rearrange the notebook safely.	Versioned SQL that changes the schema.
Seed	Filling the library with the first books.	Idempotent script inserting reference data.
GIN index	An index at the back of a book for lists.	Generalized Inverted Index for arrays/JSONB containment queries.
LLM	A program that learned language from lots of text.	Large language model predicting the next token; here Google Gemini.
Fallback	Plan B when plan A fails.	Deterministic code path used on timeout, error or invalid output.
WASM	A way to run fast code inside the browser.	WebAssembly: compiled binary code executed by the browser; MediaPipe uses it.
Landmarks / blendshapes	Dots on your face; sliders for each expression.	478 3-D face points; 52 coefficients for expressions like eyeLookOut or mouthSmile.
Yaw / pitch	Turning your head left-right / nodding up-down.	Rotation angles derived from the head transformation matrix.
TTS / STT	Computer mouth / computer ears.	Text-to-speech / speech-to-text.
Rate limiting	"Only 20 questions a minute, please."	Restricting requests per client per time window; 429 when exceeded.
Idempotent	Pressing the lift button twice still calls one lift.	Repeating an operation has the same effect as doing it once.

Cascade delete	Removing a folder removes the papers in it.	Foreign-key action deleting child rows with the parent.

Blackbox project guide · generated from the source code in `frontend/`. When the code changes, update this guide alongside it.